Тема 10. Встроенный модуль HTTP. Основы сетевого взаимодействия

node

хекслет колледж

Цель занятия:

Сформировать у студентов понимание принципов работы клиент-серверной архитектуры и научить их создавать базовый веб-сервер, способный принимать сетевые запросы и отправлять корректные ответы.

хекслет колледж

Учебные вопросы:

1. Как работает HTTP
2. Статус-коды: Язык сервера
3. Первый сервер на Node.js
4. Разбор объектов req и res

хекслет колледж

1. Как работает HTTP

HTTP (HyperText Transfer Protocol) - это главный «язык» общения в интернете.

На нем браузеры общаются с сайтами, а мобильные приложения - с серверами.

хекслет колледж

Основной принцип: Request - Response

В HTTP общение всегда строится по схеме «Запрос - Ответ».

- Клиент (браузер, мобильное приложение) всегда выступает инициатором: он задает вопрос.
- Сервер (ваша программа на Node.js) всегда ждет и отвечает.

Важно: Сервер не может сам отправить данные клиенту, если тот их не попросил (в рамках классического HTTP).

хекслет колледж

Структура сетевого сообщения

Если «разрезать» HTTP-сообщение, мы увидим три четких слоя:

A. Стартовая строка (Start Line)

Это краткое содержание того, что происходит.

- Для запроса: Метод (GET/POST) + Путь (/about) + Версия протокола.
- Для ответа: Версия протокола + Статус-код (200 OK).

кекслет колледж

Б. Заголовки (Headers)
Это «метаданные» или технические детали сообщения. Они записываются в формате Ключ: Значение.

- Пример: Content-Type: text/html сообщает браузеру, что сервер прислал веб-страницу, а не картинку.
- Пример: User-Agent рассказывает серверу, с какого устройства зашел пользователь.

хекслет колледж

В. Тело сообщения (Body)

Сами данные.

- В запросе (POST) здесь может быть логин и пароль пользователя.
- В ответе здесь находится HTML-код страницы или данные в формате JSON.

хекслет колледж

Аналогия из жизни: «Заказ в ресторане»

- **Запрос** (Request): Вы подзываете официанта (Клиент -> Сервер) и говорите: «Дай мне меню» (Метод GET, путь /menu).
- **Заголовки**: Вы уточняете: «Я говорю на русском» (Accept-Language: ru).
- **Обработка**: Официант идет на кухню (Сервер выполняет ваш код на Node.js).
- **Ответ** (Response): Официант приносит меню (Статус 200 OK).
- **Тело ответа**: Сама книжка с меню (Body).

Если меню нет: Официант возвращается и говорит «404 Not Found».
Если на кухне пожар: Официант говорит «500 Server Error».

хекслет колледж

Практическая демонстрация через терминал (curl)
Чтобы увидеть всё (и запрос, и ответ), используйте флаг -v (verbose - «подробно»).

1. Изучаем стартовую строку и заголовки введите в терминале:

curl -v http://google.com

хекслет колледж

Что вы увидите в выводе:

- Линии со знаком **>** - это Запрос (то, что curl отправил серверу).
- Линии со знаком **<** - это Ответ (то, что прислал сервер).

хекслет колледж

```
C:\Users\user>curl -v http://google.com
*   Trying 142.251.14.100:80...
* Connected to google.com (142.251.14.100) port 80 (#0)
> GET / HTTP/1.1                <-- СТАРТОВАЯ СТРОКА ЗАПРОСА
> Host: google.com              <-- ЗАГОЛОВОК (куда идем)
> User-Agent: curl/7.83.1       <-- ЗАГОЛОВОК (кто мы)
> Accept: */*
>
* Mark bundle as not supporting multiuse
```

хекслет колледж

```
* Mark bundle as not supporting multiuse
< HTTP/1.1 301 Moved Permanently          <-- СТАРТОВАЯ СТРОКА ОТВЕТА (Статус-код!)
< Location: http://www.google.com/        <-- ЗАГОЛОВОК (куда переехали)
< Content-Type: text/html; charset=UTF-8  <-- ЗАГОЛОВОК (что прислали)
< Content-Security-Policy-Report-Only: object-src 'none';base-uri 'self';script-
e-inline' https: http:;report-uri https://csp.withgoogle.com/csp/gws/other-hp
< Date: Fri, 17 Apr 2026 12:21:59 GMT
< Expires: Sun, 17 May 2026 12:21:59 GMT
< Cache-Control: public, max-age=2592000
```

хекслет колледж

Body (тело сообщения) - это всё, что идет после последней пустой строки и знака <.

```
< X-XSS-Protection: 0
< X-Frame-Options: SAMEORIGIN
<
<HTML><HEAD><meta http-equiv="content-type" content="text/html;charset=utf-8">
<TITLE>301 Moved</TITLE></HEAD><BODY>
<H1>301 Moved</H1>
The document has moved
<A HREF="http://www.google.com/">here</A>.
</BODY></HTML>
* Connection #0 to host google.com left intact

C:\Users\user>
```

это BODY

екслет колледж

итог:

- HTTP - это безобъектный протокол (stateless). Сервер не «помнит» клиента после того, как отправил ему ответ. Каждый запрос - как первый раз.

- Всё общение - это текст. Node.js просто считывает этот текст и превращает его в объекты req и res, с которыми мы работаем

хекслет колледж

2. Статус-коды: Язык сервера

Когда сервер отправляет ответ, он обязан указать числовой код. Это быстрый способ сказать клиенту: «Всё прошло успешно», «Ты ошибся» или «У меня проблемы».

Все коды делятся на 5 групп по первой цифре. Для профессиональной разработки в Node.js нужно знать «золотой стандарт» из каждой группы.

хекслет колледж

1xx - Информационные

Почти не используются в обычном веб-программировании.
Сообщают, что запрос принят и обработка продолжается.

хекслет колледж

2xx - Успех (Success) ✅

200 OK - Самый частый код. Всё прошло идеально, данные отправлены.

201 Created - Используется, когда клиент что-то отправил (например, зарегистрировал нового студента), и сервер успешно создал эту запись в базе.

кекслет колледж

3xx - Перенаправление (Redirection)

301 Moved Permanently - Страница переехала навсегда. Браузер запомнит это и в следующий раз пойдет сразу по новому адресу.

304 Not Modified - «Ничего не изменилось». Используется для кэширования. Сервер говорит браузеру: «Используй ту версию, что у тебя уже есть, она актуальна».

кекслет колледж

4xx - Ошибка клиента (Client Error) ❌

400 Bad Request - «Я тебя не понимаю». Клиент прислал данные в неверном формате (например, вместо JSON прислал просто текст).

401 Unauthorized - «Кто ты?». Требуется логин или токен доступа.

403 Forbidden - «Тебе сюда нельзя». Мы знаем, кто ты, но у тебя нет прав доступа к этой папке или файлу.

404 Not Found - Самый знаменитый код. Ресурс по указанному адресу не существует.

кекслет колледж

5xx - Ошибка сервера (Server Error) 🔥

500 Internal Server Error - «Упс, мой код сломался». Обычно это происходит, когда в вашем Node.js скрипте возникла ошибка (Exception), которую вы не обработали.

503 Service Unavailable - Сервер перегружен или на нем ведутся технические работы.

хекслет колледж

Тезисы для запоминания:

- 200 - праздник,
- 400 - ошибка в запросе,
- 500 - ошибка в коде.

Статус-код всегда идет в стартовой строке ответа, до заголовков и данных.

хекслет колледж

3. Первый сервер на Node.js

Для работы с сетью в Node.js не нужно устанавливать сторонние библиотеки. Всё необходимое уже есть во встроенном модуле **http**.

Пошаговый алгоритм создания сервера:

- **Подключение модуля**: Нам нужен инструмент для работы с протоколом.
- **Создание «слушателя»**: Мы создаем функцию, которая будет срабатывать каждый раз, когда кто-то постучится на наш адрес.
- **Назначение порта**: Мы должны сказать серверу, какой «вход» (порт) в компьютере ему занять.

хекслет колледж

Код простейшего сервера:

```
// 1. Подключаем встроенный модуль
const http = require('http');

// 2. Создаем сервер
// Мы передаем в createServer функцию-обработчик.
// Она как "дежурный", который просыпается при каждом запросе.
const server = http.createServer((req, res) => {
    // Пока что мы просто отправляем один и тот же ответ всем
    res.end('Hello! This is my first Node.js Server');
});

// 3. Заставляем сервер "слушать" определенный порт
const PORT = 3000;
server.listen(PORT, () => {
    console.log(`Сервер запущен! Адрес: http://localhost:${PORT}`);
    console.log('Нажмите Ctrl+C, чтобы остановить его.');
});
```

кекслет колледж

Разбор ключевых понятий:

Что такое localhost?

Это стандартное имя для вашего собственного компьютера.

Когда вы вводите его в браузере, запрос не уходит в интернет, а возвращается обратно вам же.

Технический адрес (IP) локальной петли - 127.0.0.1.

кекслет колледж

Что такое Порт (Port)?

- Представьте, что IP-адрес - это адрес многоквартирного дома. Тогда порт - это номер квартиры.
- На одном компьютере могут работать десятки серверов одновременно.
- Чтобы они не мешали друг другу, каждый занимает свой порт (например: 3000, 8080, 5000).

По умолчанию сайты в интернете работают на порту 80 (HTTP) или 443 (HTTPS)

хекслет колледж

Почему сервер не закрывается сам?

- В отличие от обычных скриптов, которые мы писали раньше (выполнил и вышел), сервер - это бесконечный цикл событий.
- Он входит в режим ожидания и будет работать до тех пор, пока вы не остановите его вручную в терминале или не выключите компьютер.

хекслет колледж

Демонстрация:

- Запуск: Напишите код и запустите его.
- Проверка через браузер: Откройте http://localhost:3000. Убедитесь, что страница загрузилась.
- Проверка через терминал: Откройте второе окно терминала и введите:

```
curl http://localhost:3000
```

хекслет колледж

Остановка:

- Нажмите Ctrl + C в терминале с сервером.
- Попробуйте обновить страницу в браузере - она выдаст ошибку «Не удается получить доступ к сайту».
- Это доказывает, что страницу генерировал именно ваш код, а не «магия интернета»

хекслет колледж

4. Разбор объектов req и res

Когда мы вызываем:

```
http.createServer((req, res) => { ... })
```

Node.js автоматически создает два мощных объекта для каждого входящего запроса.

Это «вход» и «выход» нашей программы.

хекслет колледж

А. Объект **req** (IncomingMessage) - «Вход»

req (сокращение от Request) - это объект, который содержит всю информацию о том, кто к нам пришел и чего он хочет.

Это «письмо», которое **прислал клиент.**

хекслет колледж

Основные свойства, которые нам нужны:
1. req.url - строка с адресом запроса.
Пример: Если пользователь зашел на http://localhost:3000/contacts, то req.url будет равен "/contacts". На основе этого мы будем решать, какую страницу показать.

2. req.method - какой тип запроса пришел.
Пример: GET (просто просмотр) или POST (отправка данных из формы).

3. req.headers - объект со всеми техническими заголовками.
Пример: req.headers['user-agent'] подскажет нам, с какого браузера зашел человек.

кекслет колледж

Б. Объект res (ServerResponse) - «Выход»

res (сокращение от Response) - это объект, через который мы формируем наш ответ. Это «пустой бланк», который нам нужно заполнить и **отправить обратно**.

хекслет колледж

Основные методы и свойства:

1. res.statusCode - тот самый статус-код (по умолчанию 200).
2. res.setHeader('Content-Type', 'text/html') - добавление заголовков по одному.
3. res.writeHead(status, { headers }) - «тяжелая артиллерия». Позволяет одной командой задать и статус, и все заголовки сразу.

- Важно: После вызова этого метода заголовки уже нельзя изменить.

4. res.write(data) - добавляет порцию данных в тело ответа. Можно вызывать многократно (например, передавать части длинного текста).
5. res.end(data) - самый важный метод. Он говорит серверу: «Всё, ответ сформирован, запечатывай конверт и отправляй!».

- Если не вызвать res.end(), браузер клиента будет вечно крутить индикатор загрузки, ожидая окончания сообщения.

кекслет колледж

Сравнение:

Свойство / Объект	req (Запрос)	res (Ответ)
Роль	Информация от клиента	Инструменты сервера
Аналогия	Заказ гостя в ресторане	Поднос с едой от официанта
Что читаем/пишем	Читаем url, method	Пишем statusCode, body
Главный метод	—	res.end() (обязателен)

кекслет колледж

Пример: «Шпионский сервер»

```
const http = require('http');

const server = http.createServer((req, res) => {
    // Читаем данные из req
    const userAgent = req.headers['user-agent'];
    const url = req.url;

    res.writeHead(200, { 'Content-Type': 'text/html; charset=utf-8' });

    // Формируем ответ на основе данных запроса
    res.write(`<h1>Ваш отчет:</h1>`);
    res.write(`<p>Вы запрашивали страницу: <b>${url}</b></p>`);
    res.write(`<p>Ваш браузер: ${userAgent}</p>`);
    res.end();
});
```

екслет колледж

```
// 3. Заставляем сервер "слушать" определенный порт
const PORT = 3000;
server.listen(PORT, () => {
    console.log(`Сервер запущен! Адрес: http://localhost:${PORT}`);
    console.log('Нажмите Ctrl+C, чтобы остановить его.');
});
```

localhost:3000

Ваш отчет:

Вы запрашивали страницу: /

Ваш браузер: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/147.0.0.0 Safari/537.36

кекслет колледж

Свойства req и методы res

Работа сервера - это всегда обработка входящего объекта и наполнение исходящего.

хекслет колледж

Объект req (IncomingMessage) - Что мы знаем о госте?

Этот объект доступен нам только для чтения. Мы не можем изменить URL запроса, мы можем только на него среагировать.

1. req.url

- Что это: Строка, содержащая всё, что идет в адресе после домена.
- Зачем: Это основа роутинга (маршрутизации).
- Пример: Если адрес http://localhost:3000/users?id=5, то req.url будет /users?id=5.

кекслет колледж

2. req.method

- Что это: Тип действия (GET, POST, PUT, DELETE).
- Зачем: Позволяет различать намерения. На один и тот же адрес /profile можно зайти (GET - показать профиль) или отправить форму (POST - обновить данные).

3. req.headers

- Что это: Объект, где ключи - названия заголовков (всегда в нижнем регистре).
- Зачем: Проверка авторизации, определение типа данных (Content-Type запроса) или языка пользователя.

кекслет колледж

Объект res (ServerResponse) - Как мы готовим ответ?

Этот объект - наш «конструктор». Мы вызываем его методы, чтобы сформировать пакет данных.

1. res.writeHead(status, [headers])

- Что это: Метод, который отправляет стартовую строку и заголовки разом.
- Важно: Это «точка невозврата». Как только вы вызвали writeHead, вы больше не можете менять заголовки или статус-код.
- Пример: res.writeHead(404, { 'Content-Type': 'text/plain' });

2. res.write(chunk)

- Что это: Отправка части (куска) тела ответа.
- Зачем: Если мы хотим отправлять данные по частям (например, сначала шапку сайта, потом контент, потом подвал). Можно вызывать много раз.

кекслет колледж

3. res.end([data])

- Что это: Сигнал о завершении ответа.
- Зачем: Без него соединение останется открытым. Если передать в него data, это будет равносильно вызову res.write(data) и последующему закрытию.

4. res.statusCode

- Что это: Свойство для установки кода (если не используется writeHead).
- Пример: res.statusCode = 404;

хекслет колледж

Порядок вызовов (Важно!)

Нет смысла записывать что-то после того, как «письмо ушло».

В HTTP порядок строгий:

1. **Установка статуса** и заголовков (res.statusCode или res.writeHead).
2. **Запись тела** (res.write).
3. **Завершение** (res.end).

Запрещенный прием: Если вы напишете res.end(), а на следующей строке попытаетесь сделать res.write(), Node.js выдаст ошибку: "Error: write after end". Это как пытаться доложить что-то в посылку, которую почтальон уже унес.

хекслет колледж

Шпаргалка:

Задача	Инструмент
Узнать, какую страницу хочет юзер	req.url
Понять, отправляет он данные или получает	req.method
Указать, что мы отправляем именно HTML	res.setHeader('Content-Type', 'text/html')
Сказать браузеру, что страницы нет	res.statusCode = 404
Отправить финальный кусок данных и закрыть связь	res.end('Goodbye!')

кеслет колледж

Content-Type

Content-Type - это заголовок ответа, который сообщает клиенту (браузеру или приложению), какой тип данных пришел в теле сообщения.

Без этого заголовка браузер может начать «гадать» и часто ошибается. Мы, как бэкенд-разработчики, должны четко указывать формат.

хекслет колледж

Самые важные типы (MIME-типы):

- text/plain - обычный текст. Теги <h1> будут отображены как текст, а не как заголовок.
- text/html - HTML-документ. Браузер отрисует все теги и создаст DOM-дерево.
- application/json - данные в формате JSON (стандарт для API и мобильных приложений).
- image/jpeg или image/png - изображения.

хекслет колледж

Проблема кодировки (UTF-8)

Если вы отправите кириллицу (русские буквы), но не укажете кодировку, пользователь увидит «кракозябры» (ромбики или странные символы).

Правильный формат:

```
res.writeHead(200, { 'Content-Type': 'text/html; charset=utf-8' });
```

хекслет колледж

Итоги лекции:

- HTTP - это текст. Запрос от клиента, ответ от сервера.
- Порт - «квартира» в компьютере, где работает наш сервер (3000).
- req - читаем, что хочет клиент (url, method).
- res - пишем ответ (statusCode, writeHead, end).
- Content-Type - объясняем браузеру, что именно мы прислали.

хекслет колледж

Контрольные вопросы

- Кто всегда является инициатором общения в протоколе HTTP - клиент или сервер?
- Программист написал код, который выводит «Ошибка базы данных», но забыл указать статус-код. Какой статус увидит браузер в консоли Network?
- В чем разница между методами res.write() и res.end()? Можно ли вызвать res.write() после того, как был вызван res.end()?
- Какое свойство объекта req поможет нам понять, хочет ли пользователь открыть страницу «О нас» или «Главную»?
- Что увидит пользователь в браузере, если вы отправите строку <h1>Hello</h1>, указав заголовок Content-Type: text/plain?
- Почему при отправке русского текста важно добавлять ; charset=utf-8 в заголовок Content-Type?

кекслет колледж